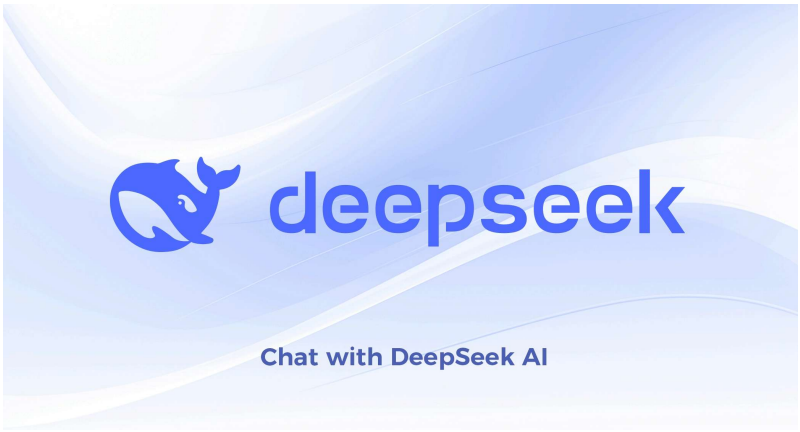
MongoDB Express Concepts Explained - DeepSeek



I'll explain MongoDB with Express concepts step by step, from basics to integration.

1. MongoDB Basics

What is MongoDB?

- **NoSQL database** (non-relational)
- Stores data in **JSON-like documents** (BSON format)
- **Schema-flexible** - no fixed table structure
- Uses collections instead of tables, documents instead of rows

Key MongoDB Concepts:

javascript

```
Database: "ecommerce"
  └─ Collection: "users"
      ├─ Document: {name: "John", age: 30}
      └─ Document: {name: "Jane", age: 25}
```

Common MongoDB Operations:

javascript

```
db.users.insertOne({name: "John", email: "john@example.com"})

db.users.find({name: "John"})

db.users.updateOne({name: "John"}, {$set: {age: 31}})

db.users.deleteOne({name: "John"})
```

2. Express.js Basics

What is Express?

- **Web framework** for Node.js
- Simplifies building REST APIs and web servers
- Provides routing, middleware, HTTP utilities

Basic Express Server:

javascript

```
const express = require('express');
const app = express();
const PORT = 3000;


app.use(express.json());


app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

3. Mongoose (ODM for MongoDB)

What is Mongoose?

- **Object Data Modeling** library for MongoDB

- Provides schema validation and type casting
- Creates a structured way to interact with MongoDB

Mongoose Schema:

javascript

```
const mongoose = require('mongoose');


const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  age: {
    type: Number,
    min: 0
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});


const User = mongoose.model('User', userSchema);
```

4. Connecting Express with MongoDB

Connection Setup:

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();


mongoose.connect('mongodb://localhost:27017/myapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('MongoDB connected'))
.catch(err => console.log('Connection error:', err));

app.use(express.json());

app.listen(3000, () => {
  console.log('Server running');
});
```

5. CRUD Operations with Express + MongoDB

Create (POST):

javascript

```
app.post('/api/users', async (req, res) => {
  try {
    const { name, email, age } = req.body;

    const user = new User({
      name,
      email,
      age
    });

    const savedUser = await user.save();
    res.status(201).json(savedUser);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});
```

Read (GET):

javascript

```
app.get('/api/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});


app.get('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: 'User
not found' });
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

Update (PUT/PATCH):

javascript

```
app.put('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!user) return res.status(404).json({ message: 'User
not found' });
    res.json(user);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});
```

Delete (DELETE):

javascript

```
app.delete('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);
    if (!user) return res.status(404).json({ message: 'User
not found' });
    res.json({ message: 'User deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

6. Advanced Query Operations

Filtering and Sorting:

javascript

```
app.get('/api/users', async (req, res) => {
  try {
    const { minAge, maxAge, sortBy, limit } = req.query;

    let query = User.find();


    if (minAge || maxAge) {
      const ageFilter = {};
      if (minAge) ageFilter.$gte = parseInt(minAge);
      if (maxAge) ageFilter.$lte = parseInt(maxAge);
      query = query.where('age', ageFilter);
    }


    if (sortBy) {
      const sortOrder = sortBy.startsWith('-') ? -1 : 1;
      const sortField = sortBy.replace(/^-/, '');
      query = query.sort({ [sortField]: sortOrder });
    }


    if (limit) {
      query = query.limit(parseInt(limit));
    }

    const users = await query;
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

7. Relationships (Population)

Defining Related Schemas:

javascript

```
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }
});

const Post = mongoose.model('Post', postSchema);


app.get('/api/posts', async (req, res) => {
  const posts = await Post.find().populate('author', 'name
email');
  res.json(posts);
});
```

8. Middleware in Mongoose

Pre and Post Hooks:

javascript

```
userSchema.pre('save', function(next) {

  if (this.isModified('password')) {
    this.password = hash(this.password);
  }
  next();
});

userSchema.post('save', function(doc, next) {
  console.log(`${doc.name} was saved`);
  next();
});
```

9. Error Handling

Comprehensive Error Handling:

javascript

```
app.use((err, req, res, next) => {
  console.error(err.stack);

  if (err.name === 'ValidationError') {
    return res.status(400).json({
      message: 'Validation Error',
      errors: Object.values(err.errors).map(e => e.message)
    });
  }

  if (err.code === 11000) {
    return res.status(400).json({
      message: 'Duplicate key error'
    });
  }

  res.status(500).json({ message: 'Something went wrong!' });
});
```

10. Complete Example Structure

Project Structure:

text

```
my-app/
├── models/
│   ├── User.js
│   └── Post.js
├── controllers/
│   ├── userController.js
│   └── postController.js
├── routes/
│   ├── userRoutes.js
│   └── postRoutes.js
├── middleware/
│   └── auth.js
├── config/
│   └── database.js
└── server.js
```

Modular Route Example:

javascript

```
const router = require('express').Router();
const User = require('../models/User');

router.get('/', async (req, res) => {
  const users = await User.find();
  res.json(users);
});

router.post('/', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).json(user);
});

module.exports = router;


const userRoutes = require('./routes/userRoutes');
app.use('/api/users', userRoutes);
```

Key Takeaways:

1. **MongoDB** stores flexible, JSON-like documents
2. **Express** handles HTTP requests and routing
3. **Mongoose** bridges Express and MongoDB with schemas
4. **CRUD operations** form the backbone of any API
5. **Error handling** is crucial for production apps
6. **Modular structure** keeps code organized

This combination (MERN stack's 'M' and 'E') is powerful for building full-stack JavaScript applications with efficient data persistence.

basics set up

MongoDB with Express - Basic Setup Guide

Step-by-Step Setup from Scratch

Step 1: Install Node.js and npm

First, ensure Node.js is installed on your system:

bash

```
node --version
npm --version
```

Step 2: Create Project Directory

bash

```
mkdir my-mongodb-express-app
cd my-mongodb-express-app
```

Step 3: Initialize npm Project

bash

```
npm init -y
```

This creates `package.json` file.

Step 4: Install Required Dependencies

bash

```
npm install express mongodb mongoose

npm install -D nodemon
```

What each package does:

- `express` - Web framework
- `mongodb` - MongoDB driver for Node.js
- `mongoose` - ODM for MongoDB (easier than raw mongodb)

- `nodemon` - Auto-restarts server on changes

Step 5: Install MongoDB

Option A: Local MongoDB Installation

- **Windows**: Download from MongoDB Community Server
- **Mac**: `brew install mongodb-community`
- **Linux**: `sudo apt-get install mongodb`

Option B: MongoDB Atlas (Cloud - Easiest)

1. Go to MongoDB Atlas
2. Create free account
3. Create a cluster (free tier)
4. Get connection string

Step 6: Create Basic Server Setup

Create `server.js`:

javascript

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
const PORT = process.env.PORT || 3000;


app.use(express.json());
app.use(express.urlencoded({ extended: true }));


app.get('/', (req, res) => {
  res.json({ message: 'Welcome to MongoDB + Express API' });
});


app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Step 7: Connect to MongoDB

Create `config/database.js`:

javascript

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {

    const conn = await
mongoose.connect('mongodb://localhost:27017/myapp', {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });

    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error('MongoDB connection error:', error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Update `server.js` with Database Connection:

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const connectDB = require('./config/database');

const app = express();


connectDB();


app.use(express.json());


const PORT = 3000;


mongoose.connection.once('open', () => {
  console.log('Database connected, starting server...');
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
});


app.get('/', (req, res) => {
  res.json({ message: 'API is working!' });
});
```

Step 8: Create a Model (Schema)

Create `models/User.js`:

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true
  },
  age: {
    type: Number,
    min: 0,
    max: 120
  },
  isActive: {
    type: Boolean,
    default: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('User', userSchema);
```

Step 9: Create Routes and Controllers

Create `controllers/userController.js`:

javascript

```
const User = require('../models/User');


exports.createUser = async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json({ success: true, data: user });
  } catch (error) {
    res.status(400).json({ success: false, error:
error.message });
  }
};


exports.getUsers = async (req, res) => {
  try {
    const users = await User.find();
    res.status(200).json({ success: true, count: users.length,
data: users });
  } catch (error) {
    res.status(500).json({ success: false, error:
error.message });
  }
};


exports.getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ success: false, error:
'User not found' });
    }
    res.status(200).json({ success: true, data: user });
  } catch (error) {
    res.status(500).json({ success: false, error:
error.message });
  }
};
```

Create `routes/userRoutes.js`:

javascript

```
const express = require('express');
const router = express.Router();
const { createUser, getUsers, getUser } =
require('../controllers/userController');


router.post('/users', createUser);
router.get('/users', getUsers);
router.get('/users/:id', getUser);

module.exports = router;
```

Step 10: Update Server with Routes

Final `server.js`:

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const connectDB = require('./config/database');
const userRoutes = require('./routes/userRoutes');

const app = express();


connectDB();


app.use(express.json());
app.use(express.urlencoded({ extended: true }));


app.use('/api', userRoutes);


app.get('/', (req, res) => {
  res.json({
    message: 'MongoDB + Express API',
    endpoints: {
      createUser: 'POST /api/users',
      getUsers: 'GET /api/users',
      getUser: 'GET /api/users/:id'
    }
  });
});


app.use('*', (req, res) => {
  res.status(404).json({ success: false, error: 'Route not
found' });
});


const PORT = process.env.PORT || 3000;

mongoose.connection.once('open', () => {
  console.log('✅ MongoDB connected');
  app.listen(PORT, () => {
    console.log(`✅ Server running on
http://localhost:${PORT}`);
  });
});

mongoose.connection.on('error', (err) => {
  console.error('❌ MongoDB connection error:', err);
});
```

Step 11: Add Package.json Scripts

Update `package.json`:

json

```
{
  "name": "my-mongodb-express-app",
  "version": "1.0.0",
  "description": "Basic MongoDB Express Setup",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.5.0",
    "mongodb": "^6.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Step 12: Create Environment Variables (Optional but Recommended)

Install dotenv:

bash

```
npm install dotenv
```

Create `.env` file:

env

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/myapp
```

Update `config/database.js`:

javascript

```
const mongoose = require('mongoose');
require('dotenv').config();

const connectDB = async () => {
  try {
    const conn = await
mongoose.connect(process.env.MONGODB_URI ||
'mongodb://localhost:27017/myapp', {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error('Error:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create `.gitignore`:

text

```
node_modules/
.env
.DS_Store
```

Step 13: Run the Application

bash

```
npm run dev


npm start
```

Step 14: Test Your API

Using **Postman** or **curl**:

bash

```
curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"John Doe","email":"john@example.com","age":25}'


curl http://localhost:3000/api/users


curl http://localhost:3000/api/users/USER_ID_HERE
```

Quick Setup with One Command (Alternative)

If you want an even faster setup, create a setup script:

Create `setup.sh` (Linux/Mac) or `setup.bat` (Windows):

bash

```
#!/bin/bash
mkdir my-app && cd my-app
npm init -y
npm install express mongoose mongodb
npm install -D nodemon
mkdir config models controllers routes
touch server.js config/database.js models/User.js
controllers/userController.js routes/userRoutes.js
echo "Setup complete! Run 'npm run dev' to start"
```

Troubleshooting Common Issues

Issue 1: MongoDB Connection Refused

bash

```


net start MongoDB


brew services start mongodb-community


sudo systemctl start mongodb
```

Issue 2: Port Already in Use

bash

```
PORT=3001
```

Issue 3: EADDRINUSE Error

bash

```
netstat -ano | findstr :3000
taskkill /PID <PID> /F

lsof -i :3000
kill -9 <PID>
```

Folder Structure After Setup:

text

```
my-mongodb-express-app/
├── node_modules/
├── config/
│   └── database.js
├── models/
│   └── User.js
├── controllers/
│   └── userController.js
├── routes/
│   └── userRoutes.js
├── .env
├── .gitignore
├── package.json
├── package-lock.json
└── server.js
```

That's it! You now have a working MongoDB + Express application ready for development!

creating our model chat

Creating Models in MongoDB with Express - Complete Guide

What is a Model?

A **model** in Mongoose is a wrapper for the schema that provides an interface to the database for CRUD operations. Think of it as a **blueprint** for creating and querying documents in a specific MongoDB collection.

Basic Model Structure

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({

});

const User = mongoose.model('User', userSchema);

module.exports = User;
```

Step-by-Step Model Creation

1. Simple User Model

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

module.exports = mongoose.model('User', userSchema);
```

2. User Model with Field Types & Validation

javascript

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({

  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters'],
    maxlength: [50, 'Name cannot exceed 50 characters']
  },


  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
'Please enter a valid email']
  },


  age: {
    type: Number,
    min: [0, 'Age cannot be negative'],
    max: [120, 'Age cannot exceed 120'],
    validate: {
      validator: Number.isInteger,
      message: 'Age must be an integer'
    }
  },


  isActive: {
    type: Boolean,
    default: true
  },


  createdAt: {
    type: Date,
    default: Date.now,
    immutable: true
  },


  hobbies: [{
```

```
    type: String,
    trim: true
  }],


  address: {
    street: String,
    city: String,
    zipCode: String,
    country: {
      type: String,
      default: 'India'
    }
  }
});

module.exports = mongoose.model('User', userSchema);
```

Different Types of Models

3. Product Model (E-commerce)

javascript

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product name is required'],
    unique: true,
    trim: true
  },

  price: {
    type: Number,
    required: true,
    min: [0, 'Price cannot be negative'],
    get: v => (v / 100).toFixed(2),
    set: v => v * 100
  },

  category: {
    type: String,
    enum: ['Electronics', 'Clothing', 'Books', 'Food',
'Other'],
    required: true
  },

  inStock: {
    type: Boolean,
    default: true
  },

  quantity: {
    type: Number,
    default: 0,
    min: 0
  },

  tags: [String],

  ratings: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    },
    rating: {
      type: Number,
      min: 1,
      max: 5
    },
    comment: String,
```

```
    date: {
      type: Date,
      default: Date.now
    }
  }]
}, {
  timestamps: true
});

module.exports = mongoose.model('Product', productSchema);
```

4. Blog Post Model (with Relationships)

javascript

```
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },

  slug: {
    type: String,
    required: true,
    unique: true,
    lowercase: true
  },

  content: {
    type: String,
    required: true
  },

  excerpt: {
    type: String,
    maxlength: 200
  },

  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },

  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category'
  },

  tags: [{
    type: String,
    lowercase: true
  }],

  views: {
    type: Number,
    default: 0
  },

  likes: [{
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'User'
  }],

  comments: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    },
    text: {
      type: String,
      required: true
    },
    date: {
      type: Date,
      default: Date.now
    }
  }],

  status: {
    type: String,
    enum: ['draft', 'published', 'archived'],
    default: 'draft'
  },

  publishedAt: Date
}, {
  timestamps: true
});


postSchema.index({ title: 'text', content: 'text' });


postSchema.virtual('readingTime').get(function() {
  const wordsPerMinute = 200;
  const wordCount = this.content.split(/\s+/).length;
  return Math.ceil(wordCount / wordsPerMinute);
});

module.exports = mongoose.model('Post', postSchema);
```

5. Category Model

javascript

```
const mongoose = require('mongoose');

const categorySchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },

  description: String,

  parent: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category'
  },

  slug: {
    type: String,
    required: true,
    unique: true
  }
});

module.exports = mongoose.model('Category', categorySchema);
```

Advanced Model Features

6. Model with Middleware (Hooks)

javascript

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6,
    select: false
  },
  name: String,
  resetPasswordToken: String,
  resetPasswordExpire: Date
});


userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();

  try {
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
  } catch (error) {
    next(error);
  }
});


userSchema.pre('remove', async function(next) {
  await Post.deleteMany({ author: this._id });
  next();
});


userSchema.methods.comparePassword = async
function(candidatePassword) {
  return await bcrypt.compare(candidatePassword,
this.password);
};


userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() });
```

```
};


userSchema.query.active = function() {
  return this.where({ isActive: true });
};

module.exports = mongoose.model('User', userSchema);
```

7. Model with Custom Validation

javascript

```
const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({
  orderNumber: {
    type: String,
    unique: true,
    default: function() {
      return 'ORD-' + Date.now() + '-' +
Math.random().toString(36).substr(2, 5);
    }
  },

  items: [{
    product: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Product',
      required: true
    },
    quantity: {
      type: Number,
      required: true,
      min: 1,
      validate: {
        validator: Number.isInteger,
        message: 'Quantity must be integer'
      }
    },
    price: Number
  }],

  totalAmount: {
    type: Number,
    required: true,
    validate: {
      validator: function(value) {
        return value >= 0;
      },
      message: 'Total amount cannot be negative'
    }
  },

  status: {
    type: String,
    enum: ['pending', 'processing', 'shipped', 'delivered',
'cancelled'],
    default: 'pending'
  },

  customer: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, {
  timestamps: true
});


orderSchema.index({ customer: 1, status: 1 });

module.exports = mongoose.model('Order', orderSchema);
```

Using Models in Controllers

Complete CRUD with Model

javascript

```
const User = require('../models/User');


exports.createUser = async (req, res) => {
  try {
    const user = new User(req.body);
    const savedUser = await user.save();
    res.status(201).json({
      success: true,
      data: savedUser
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
};


exports.getUsers = async (req, res) => {
  try {

    let query = User.find();


    if (req.query.active) {
      query = query.where('isActive').equals(req.query.active
=== 'true');
    }


    if (req.query.minAge || req.query.maxAge) {
      const ageFilter = {};
      if (req.query.minAge) ageFilter.$gte =
parseInt(req.query.minAge);
      if (req.query.maxAge) ageFilter.$lte =
parseInt(req.query.maxAge);
      query = query.where('age', ageFilter);
    }


    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;
    const skip = (page - 1) * limit;

    query = query.skip(skip).limit(limit);
```

```
    if (req.query.sort) {
      const sortField = req.query.sort.startsWith('-') ?
req.query.sort.substring(1) : req.query.sort;
      const sortOrder = req.query.sort.startsWith('-') ? -1 :
1;
      query = query.sort({ [sortField]: sortOrder });
    }


    const users = await query;
    const total = await
User.countDocuments(query._conditions);

    res.json({
      success: true,
      count: users.length,
      total,
      pagination: {
        page,
        limit,
        totalPages: Math.ceil(total / limit)
      },
      data: users
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
};


exports.getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id)
      .select('-__v')
      .populate('posts');

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
```

```
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
};


exports.updateUser = async (req, res) => {
  try {

    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
};


exports.deleteUser = async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      message: 'User deleted successfully'
```

```
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
};


exports.findByEmail = async (req, res) => {
  try {
    const user = await User.findByEmail(req.params.email);
    res.json({ success: true, data: user });
  } catch (error) {
    res.status(500).json({ success: false, error:
error.message });
  }
};


exports.getActiveUsers = async (req, res) => {
  try {
    const users = await User.find().active();
    res.json({ success: true, data: users });
  } catch (error) {
    res.status(500).json({ success: false, error:
error.message });
  }
};
```

Model Relationships Cheat Sheet

javascript

```
const profileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  bio: String
});


const userSchema = new mongoose.Schema({
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Post'
}]
});


const postSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User'
}
});


const userSchema = new mongoose.Schema({
  followers: [{ type: mongoose.Schema.Types.ObjectId, ref:
'User' }],
  following: [{ type: mongoose.Schema.Types.ObjectId, ref:
'User' }]
});
```

Best Practices for Models

1. **Always validate data** using built-in or custom validators
2. **Use meaningful field names** and add comments for complex logic
3. **Index frequently queried fields** for better performance
4. **Use timestamps** option for tracking creation/updates
5. **Keep models focused** - one model per file
6. **Use middleware** for cross-cutting concerns (password hashing, slug generation)
7. **Define virtuals** for computed properties
8. **Use select: false** for sensitive fields (passwords, tokens)

Quick Model Generator Template

javascript

```
const mongoose = require('mongoose');

const modelNameSchema = new mongoose.Schema({

  field1: {
    type: String,
    required: true
  },
  field2: {
    type: Number,
    default: 0
  }
}, {
  timestamps: true
});


modelNameSchema.index({ field1: 1 });


modelNameSchema.methods.customMethod = function() {
  return this.field1.toUpperCase();
};


modelNameSchema.statics.customStatic = function() {
  return this.find({});
};

module.exports = mongoose.model('ModelName', modelNameSchema);
```

Now you're ready to create powerful, validated models for your MongoDB + Express application!

initialize database

Initializing Database in MongoDB with Express - Complete Guide

What is Database Initialization?

Database initialization is the process of **setting up your database** when your application starts - including connecting, creating indexes, seeding initial data, and ensuring everything is ready for your app to use.

Complete Database Initialization Setup

Step 1: Database Configuration File

`config/database.js` - Complete Setup

javascript

```
const mongoose = require('mongoose');
require('dotenv').config();

class Database {
  constructor() {
    this.isConnected = false;
  }


  async connect() {
    if (this.isConnected) {
      console.log('Database already connected');
      return;
    }

    try {
      const conn = await
mongoose.connect(process.env.MONGODB_URI ||
'mongodb://localhost:27017/myapp', {
        useNewUrlParser: true,
        useUnifiedTopology: true,
        serverSelectionTimeoutMS: 5000,
        heartbeatFrequencyMS: 2000,
      });

      this.isConnected = true;
      console.log(`✅ MongoDB Connected:
${conn.connection.host}:${conn.connection.port}`);
      console.log(`📁 Database Name:
${conn.connection.name}`);


      this.setupEventListeners();

      return conn;
    } catch (error) {
      console.error('❌ MongoDB Connection Error:',
error.message);
      this.isConnected = false;
      throw error;
    }
  }


  setupEventListeners() {
    mongoose.connection.on('disconnected', () => {
      console.log('⚠️ MongoDB disconnected');
      this.isConnected = false;
    });
```

```
    mongoose.connection.on('reconnected', () => {
      console.log('✅ MongoDB reconnected');
      this.isConnected = true;
    });

    mongoose.connection.on('error', (err) => {
      console.error('❌ MongoDB error:', err);
    });


    process.on('SIGINT', async () => {
      await this.disconnect();
      process.exit(0);
    });
  }


  async disconnect() {
    if (!this.isConnected) return;

    try {
      await mongoose.disconnect();
      this.isConnected = false;
      console.log('✅ MongoDB disconnected gracefully');
    } catch (error) {
      console.error('❌ Error disconnecting MongoDB:', error);
    }
  }


  getConnectionStatus() {
    return {
      isConnected: this.isConnected,
      readyState: mongoose.connection.readyState,
      readyStateText:
this.getReadyStateText(mongoose.connection.readyState)
    };
  }

  getReadyStateText(state) {
    const states = {
      0: 'disconnected',
      1: 'connected',
      2: 'connecting',
      3: 'disconnecting'
    };
    return states[state] || 'unknown';
  }
}
```

```
module.exports = new Database();
```

Step 2: Database Initialization with Indexes

`config/initDatabase.js` - Initialization Logic

javascript

```
const User = require('../models/User');
const Product = require('../models/Product');
const Post = require('../models/Post');

class DatabaseInitializer {


  static async initialize() {
    console.log('🔄 Initializing database...');

    try {

      await this.createIndexes();


      await this.seedInitialData();


      await this.setupDatabaseHooks();

      console.log('✅ Database initialization complete');
    } catch (error) {
      console.error('❌ Database initialization failed:',
error);
      throw error;
    }
  }


  static async createIndexes() {
    console.log('📇 Creating database indexes...');


    await User.collection.createIndex({ email: 1 }, { unique:
true });
    await User.collection.createIndex({ name: 'text' });
    await User.collection.createIndex({ createdAt: -1 });


    await Product.collection.createIndex({ name: 'text' });
    await Product.collection.createIndex({ price: 1 });
    await Product.collection.createIndex({ category: 1, price:
-1 });


    await Post.collection.createIndex({ title: 'text',
content: 'text' });
    await Post.collection.createIndex({ author: 1, createdAt:
-1 });
    await Post.collection.createIndex({ status: 1,
```

```
publishedAt: -1 });

    console.log('✅ Indexes created successfully');
  }


  static async seedInitialData() {
    console.log('🌱 Checking for existing data...');


    const userCount = await User.countDocuments();
    if (userCount === 0) {
      console.log('Seeding users...');
      await User.insertMany(initialData.users);
    }


    const productCount = await Product.countDocuments();
    if (productCount === 0) {
      console.log('Seeding products...');
      await Product.insertMany(initialData.products);
    }


    const postCount = await Post.countDocuments();
    if (postCount === 0) {
      console.log('Seeding posts...');
      await Post.insertMany(initialData.posts);
    }

    console.log('✅ Initial data seeding complete');
  }


  static async setupDatabaseHooks() {
    console.log('🔧 Setting up database hooks...');


    User.schema.pre('save', function(next) {
      this.updatedAt = new Date();
      next();
    });


    Post.schema.pre('save', function(next) {
      if (this.isModified('title')) {
        this.slug = this.title
          .toLowerCase()
          .replace(/[^a-z0-9]+/g, '-')
          .replace(/^-|-$/g, '');
      }
```

```
      next();
    });

    console.log('✅ Database hooks configured');
  }
}


const initialData = {
  users: [
    {
      name: 'Admin User',
      email: 'admin@example.com',
      age: 30,
      isActive: true,
      role: 'admin'
    },
    {
      name: 'John Doe',
      email: 'john@example.com',
      age: 25,
      isActive: true,
      role: 'user'
    },
    {
      name: 'Jane Smith',
      email: 'jane@example.com',
      age: 28,
      isActive: true,
      role: 'user'
    }
  ],

  products: [
    {
      name: 'Laptop',
      price: 999.99,
      category: 'Electronics',
      inStock: true,
      quantity: 10,
      tags: ['computer', 'tech']
    },
    {
      name: 'Coffee Mug',
      price: 15.99,
      category: 'Home',
      inStock: true,
      quantity: 50,
      tags: ['kitchen', 'drinkware']
    },
    {
```

```
      name: 'Book: JavaScript Guide',
      price: 29.99,
      category: 'Books',
      inStock: true,
      quantity: 100,
      tags: ['programming', 'education']
    }
  ],

  posts: [
    {
      title: 'Welcome to Our Blog',
      content: 'This is our first blog post...',
      status: 'published',
      views: 100
    },
    {
      title: 'Getting Started with MongoDB',
      content: 'Learn how to use MongoDB...',
      status: 'published',
      views: 250
    }
  ]
};

module.exports = DatabaseInitializer;
```

Step 3: Connection with Retry Logic

`config/databaseWithRetry.js` - **Production Ready**

javascript

```
const mongoose = require('mongoose');
require('dotenv').config();

class DatabaseConnection {
  constructor() {
    this.retryCount = 0;
    this.maxRetries = 5;
    this.retryDelay = 5000;
  }

  async connectWithRetry() {
    try {
      await mongoose.connect(process.env.MONGODB_URI, {
        useNewUrlParser: true,
        useUnifiedTopology: true,
        serverSelectionTimeoutMS: 5000,
        socketTimeoutMS: 45000,
      });

      console.log('✅ MongoDB connected successfully');
      this.retryCount = 0;
      return true;

    } catch (error) {
      console.error(`❌ MongoDB connection failed (attempt
${this.retryCount + 1}/${this.maxRetries}):`, error.message);

      if (this.retryCount < this.maxRetries) {
        this.retryCount++;
        console.log(`🔄 Retrying in ${this.retryDelay/1000}
seconds...`);
        setTimeout(() => this.connectWithRetry(),
this.retryDelay);
      } else {
        console.error('❌ Max retries reached. Could not
connect to MongoDB.');
        process.exit(1);
      }
    }
  }

  async initializeAndConnect() {
    await this.connectWithRetry();


    mongoose.connection.on('error', (err) => {
      console.error('MongoDB error:', err);
    });

    mongoose.connection.on('disconnected', () => {
```

```
      console.log('MongoDB disconnected. Attempting to
reconnect...');
      this.connectWithRetry();
    });

    return mongoose.connection;
  }
}

module.exports = new DatabaseConnection();
```

Step 4: Environment Configuration

`.env` file

env

```
# Server Configuration
PORT=3000
NODE_ENV=development

# Database Configuration
MONGODB_URI=mongodb://localhost:27017/myapp
MONGODB_USER=myuser
MONGODB_PASSWORD=mypassword
MONGODB_DATABASE=myapp

# For MongoDB Atlas (Production)
#
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/m
retryWrites=true&w=majority

# Connection Pool Settings
DB_POOL_SIZE=10
DB_CONNECTION_TIMEOUT=30000
DB_SOCKET_TIMEOUT=45000
```

Step 5: Main Server with Database Initialization

`server.js` - Complete Implementation

javascript

```
const express = require('express');
const Database = require('./config/database');
const DatabaseInitializer = require('./config/initDatabase');
const userRoutes = require('./routes/userRoutes');
const productRoutes = require('./routes/productRoutes');

const app = express();
const PORT = process.env.PORT || 3000;


app.use(express.json());
app.use(express.urlencoded({ extended: true }));


app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new
Date().toISOString()}`);
  next();
});


app.get('/health', async (req, res) => {
  const dbStatus = Database.getConnectionStatus();
  res.json({
    status: 'OK',
    timestamp: new Date(),
    database: dbStatus,
    uptime: process.uptime()
  });
});


app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);


app.get('/', (req, res) => {
  res.json({
    name: 'MongoDB Express API',
    version: '1.0.0',
    endpoints: {
      health: 'GET /health',
      users: 'GET /api/users',
      products: 'GET /api/products'
    }
  });
});


app.use((err, req, res, next) => {
```

```
  console.error('Error:', err.stack);
  res.status(500).json({
    error: 'Something went wrong!',
    message: process.env.NODE_ENV === 'development' ?
err.message : undefined
  });
});


app.use('*', (req, res) => {
  res.status(404).json({ error: `Route ${req.originalUrl} not
found` });
});


const startServer = async () => {
  try {

    await Database.connect();


    await DatabaseInitializer.initialize();


    app.listen(PORT, () => {
      console.log(`🚀 Server running on
http://localhost:${PORT}`);
      console.log(`📊 Health check:
http://localhost:${PORT}/health`);
      console.log(`🌍 Environment: ${process.env.NODE_ENV ||
'development'}`);
    });

  } catch (error) {
    console.error('❌ Failed to start server:', error);
    process.exit(1);
  }
};


process.on('SIGTERM', async () => {
  console.log('SIGTERM received. Shutting down
gracefully...');
  await Database.disconnect();
  process.exit(0);
});


startServer();
```

Step 6: Database Utility Functions

`utils/databaseUtils.js` - Helper Functions

javascript

```
const mongoose = require('mongoose');

class DatabaseUtils {


  static isConnected() {
    return mongoose.connection.readyState === 1;
  }


  static async getStats() {
    const stats = {
      connectionState: mongoose.connection.readyState,
      databaseName: mongoose.connection.name,
      collections: {},
      totalSize: 0
    };

    const collections = await
mongoose.connection.db.listCollections().toArray();

    for (const collection of collections) {
      const coll =
mongoose.connection.db.collection(collection.name);
      const count = await coll.countDocuments();
      const stats = await coll.stats();

      stats.collections[collection.name] = {
        count,
        size: stats.size,
        avgObjSize: stats.avgObjSize
      };

      stats.totalSize += stats.size;
    }

    return stats;
  }


  static async backup() {
    const backup = {};
    const collections = await
mongoose.connection.db.listCollections().toArray();

    for (const collection of collections) {
      const coll =
mongoose.connection.db.collection(collection.name);
      backup[collection.name] = await coll.find({}).toArray();
    }
```

```
    return backup;
  }


  static async clearDatabase() {
    const collections = mongoose.connection.collections;

    for (const key in collections) {
      await collections[key].deleteMany({});
    }

    console.log('✅ Database cleared');
  }


  static async dropDatabase() {
    await mongoose.connection.db.dropDatabase();
    console.log('✅ Database dropped');
  }


  static async getCollectionNames() {
    const collections = await
mongoose.connection.db.listCollections().toArray();
    return collections.map(c => c.name);
  }
}

module.exports = DatabaseUtils;
```

Step 7: Database Initialization Script

`scripts/initDb.js` - Standalone Initialization Script

javascript

```
const mongoose = require('mongoose');
require('dotenv').config({ path: '../.env' });
const DatabaseInitializer = require('../config/initDatabase');

const initDatabase = async () => {
  console.log('Starting database initialization script...');

  try {

    await mongoose.connect(process.env.MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });

    console.log('Connected to MongoDB');


    await DatabaseInitializer.initialize();

    console.log('Database initialization completed
successfully');
    process.exit(0);

  } catch (error) {
    console.error('Initialization failed:', error);
    process.exit(1);
  }
};

initDatabase();
```

Run the script:

bash

```
node scripts/initDb.js
```

Step 8: Package.json Scripts

json

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "init-db": "node scripts/initDb.js",
    "seed": "node scripts/seedData.js",
    "backup": "node scripts/backupDb.js",
    "db-stats": "node scripts/getDbStats.js"
  }
}
```

Complete Initialization Flow

javascript

```
const initializeApp = async () => {
  require('dotenv').config();

  await Database.connectWithRetry();

  await DatabaseInitializer.createIndexes();

  await DatabaseInitializer.seedInitialData();

  Database.setupEventListeners();

  app.listen(PORT, () => {
    console.log(`Server ready on port ${PORT}`);
  });
};
```

Best Practices for Database Initialization

✅ DO:

- Use connection pooling
- Implement retry logic for production
- Create indexes during initialization

- Seed data only for development/staging
- Use environment variables for configuration
- Add health check endpoints
- Implement graceful shutdown

❌ DON'T:

- Don't seed production database without checks
- Don't hardcode database credentials
- Don't ignore connection errors
- Don't forget to handle disconnections
- Don't create indexes on every startup unnecessarily

Testing Database Connection

javascript

```
const Database = require('../config/database');

describe('Database Initialization', () => {
  test('Should connect to database', async () => {
    await Database.connect();
    expect(Database.isConnected).toBe(true);
  });

  test('Should have indexes created', async () => {
    const indexes = await User.collection.indexes();
    expect(indexes).toHaveLength(expect.any(Number));
  });

  test('Should have initial data', async () => {
    const userCount = await User.countDocuments();
    expect(userCount).toBeGreaterThan(0);
  });
});
```

This complete initialization system ensures your database is properly set up, connected, and ready for production use!

index route

Index Routes in Express with MongoDB - Complete Guide

What is an Index Route?

An **index route** is the main entry point or root route (`/`) of your API that typically provides:

- API information and documentation
- Available endpoints list
- API version and status
- Health check information
- Navigation links to all resources

Basic Index Route

Simple Index Route

javascript

```
app.get('/', (req, res) => {
  res.json({
    message: 'Welcome to My API',
    version: '1.0.0'
  });
});
```

Complete Index Route Implementations

1. Basic API Index Route

javascript

```
const express = require('express');
const router = express.Router();


router.get('/', (req, res) => {
  res.json({
    name: 'My MongoDB Express API',
    version: '1.0.0',
    description: 'A RESTful API for managing users, products,
and posts',
    status: 'active',
    timestamp: new Date().toISOString()
  });
});

module.exports = router;
```

2. Comprehensive API Documentation Index

javascript

```
const express = require('express');
const router = express.Router();
const packageJson = require('../package.json');

router.get('/', (req, res) => {
  res.json({

    name: packageJson.name,
    version: packageJson.version,
    description: packageJson.description,


    environment: process.env.NODE_ENV || 'development',
    timestamp: new Date().toISOString(),


    endpoints: {

      auth: {
        register: 'POST /api/auth/register',
        login: 'POST /api/auth/login',
        logout: 'POST /api/auth/logout',
        me: 'GET /api/auth/me'
      },


      users: {
        list: 'GET /api/users',
        create: 'POST /api/users',
        getOne: 'GET /api/users/:id',
        update: 'PUT /api/users/:id',
        delete: 'DELETE /api/users/:id'
      },


      products: {
        list: 'GET /api/products',
        create: 'POST /api/products',
        getOne: 'GET /api/products/:id',
        update: 'PUT /api/products/:id',
        delete: 'DELETE /api/products/:id'
      },


      posts: {
        list: 'GET /api/posts',
        create: 'POST /api/posts',
        getOne: 'GET /api/posts/:id',
        update: 'PUT /api/posts/:id',
```

```
        delete: 'DELETE /api/posts/:id'
      }
    },


    docs: {
      swagger: '/api-docs',
      postman: 'https://documenter.getpostman.com/...',
      github: packageJson.repository?.url
    },


    health: '/health'
  });
});

module.exports = router;
```

3. Dynamic Index Route with Database Stats

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const router = express.Router();
const User = require('../models/User');
const Product = require('../models/Product');
const Post = require('../models/Post');

router.get('/', async (req, res) => {
  try {

    const dbStats = {
      status: mongoose.connection.readyState === 1 ?
'connected' : 'disconnected',
      name: mongoose.connection.name,
      host: mongoose.connection.host,
      port: mongoose.connection.port
    };


    const counts = {
      users: await User.countDocuments(),
      products: await Product.countDocuments(),
      posts: await Post.countDocuments()
    };


    const recentUsers = await User.find()
      .sort({ createdAt: -1 })
      .limit(5)
      .select('name email createdAt');

    res.json({
      api: {
        name: 'MongoDB Express API',
        version: '1.0.0',
        documentation: '/api/docs'
      },
      database: dbStats,
      statistics: {
        totalRecords: counts,
        lastUpdated: new Date().toISOString()
      },
      recent: {
        users: recentUsers
      },
      endpoints: {
        users: '/api/users',
        products: '/api/products',
        posts: '/api/posts',
```

```
        auth: '/api/auth'
      },
      examples: {
        createUser: {
          method: 'POST',
          url: '/api/users',
          body: {
            name: 'John Doe',
            email: 'john@example.com',
            age: 25
          }
        }
      }
    });
  } catch (error) {
    res.status(500).json({
      error: 'Failed to fetch index data',
      message: error.message
    });
  }
});

module.exports = router;
```

4. Production-Ready Index with Documentation

javascript

```
const express = require('express');
const router = express.Router();
const fs = require('fs');
const path = require('path');


const getRouteInfo = (app) => {
  const routes = [];


  app._router.stack.forEach((middleware) => {
    if (middleware.route) {

      routes.push({
        path: middleware.route.path,
        methods: Object.keys(middleware.route.methods)
      });
    } else if (middleware.name === 'router') {

      middleware.handle.stack.forEach((handler) => {
        if (handler.route) {
          routes.push({
            path: handler.route.path,
            methods: Object.keys(handler.route.methods)
          });
        }
      });
    }
  });

  return routes;
};

router.get('/', (req, res) => {
  const apiInfo = {
    name: 'REST API',
    version: '1.0.0',
    description: 'Complete API for managing application data',


    server: {
      status: 'operational',
      time: new Date().toISOString(),
      uptime: process.uptime(),
      nodeVersion: process.version,
      platform: process.platform
    },
```

```
    database: {
      connected: mongoose.connection.readyState === 1,
      name: mongoose.connection.name || 'Not connected'
    },


    resources: {
      authentication: {
        basePath: '/api/auth',
        endpoints: [
          { method: 'POST', path: '/register', description:
'Create new account' },
          { method: 'POST', path: '/login', description:
'Login to account' },
          { method: 'POST', path: '/logout', description:
'Logout from account' },
          { method: 'GET', path: '/verify', description:
'Verify email' },
          { method: 'POST', path: '/forgot-password',
description: 'Request password reset' },
          { method: 'POST', path: '/reset-password',
description: 'Reset password' }
        ]
      },

      users: {
        basePath: '/api/users',
        endpoints: [
          { method: 'GET', path: '/', description: 'Get all
users' },
          { method: 'GET', path: '/:id', description: 'Get
user by ID' },
          { method: 'POST', path: '/', description: 'Create
new user' },
          { method: 'PUT', path: '/:id', description: 'Update
user' },
          { method: 'PATCH', path: '/:id', description:
'Partially update user' },
          { method: 'DELETE', path: '/:id', description:
'Delete user' }
        ],
        queryParameters: {
          page: 'Page number for pagination',
          limit: 'Items per page',
          sort: 'Sort field (prefix with - for descending)',
          search: 'Search term'
        }
      },

      products: {
        basePath: '/api/products',
```

```
        endpoints: [
          { method: 'GET', path: '/', description: 'Get all
products' },
          { method: 'GET', path: '/:id', description: 'Get
product by ID' },
          { method: 'POST', path: '/', description: 'Create
new product' },
          { method: 'PUT', path: '/:id', description: 'Update
product' },
          { method: 'DELETE', path: '/:id', description:
'Delete product' },
          { method: 'GET', path: '/category/:category',
description: 'Get products by category' }
        ]
      },

      posts: {
        basePath: '/api/posts',
        endpoints: [
          { method: 'GET', path: '/', description: 'Get all
posts' },
          { method: 'GET', path: '/:id', description: 'Get
post by ID' },
          { method: 'POST', path: '/', description: 'Create
new post' },
          { method: 'PUT', path: '/:id', description: 'Update
post' },
          { method: 'DELETE', path: '/:id', description:
'Delete post' }
        ]
      }
    },


    examples: {
      createUser: {
        request: {
          method: 'POST',
          url: '/api/users',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': 'Bearer <token>'
          },
          body: {
            name: 'John Doe',
            email: 'john@example.com',
            age: 25,
            role: 'user'
          }
        },
        response: {
```

```
          status: 201,
          body: {
            success: true,
            data: {
              _id: '507f1f77bcf86cd799439011',
              name: 'John Doe',
              email: 'john@example.com',
              age: 25,
              createdAt: '2024-01-01T00:00:00.000Z'
            }
          }
        }
      }
    },


    statusCodes: {
      200: 'OK - Request succeeded',
      201: 'Created - Resource created successfully',
      400: 'Bad Request - Invalid input',
      401: 'Unauthorized - Authentication required',
      403: 'Forbidden - Insufficient permissions',
      404: 'Not Found - Resource not found',
      500: 'Internal Server Error'
    },


    links: {
      self: req.protocol + '://' + req.get('host') +
req.originalUrl,
      documentation: req.protocol + '://' + req.get('host') +
'/api/docs',
      health: req.protocol + '://' + req.get('host') +
'/health',
      graphql: req.protocol + '://' + req.get('host') +
'/graphql'
    }
  };

  res.json(apiInfo);
});

module.exports = router;
```

5. API Version 2 with HATEOAS (Hypermedia)

javascript

```
const express = require('express');
const router = express.Router();

class ApiResponse {
  constructor(data, message = 'Success') {
    this.success = true;
    this.message = message;
    this.data = data;
    this.links = {};
  }

  addLink(rel, href, method) {
    this.links[rel] = { href, method };
    return this;
  }
}

router.get('/', (req, res) => {
  const response = new ApiResponse({
    name: 'Advanced API',
    version: '2.0.0',
    features: ['Authentication', 'CRUD Operations', 'Search',
'Pagination', 'Filtering']
  });


  response
    .addLink('self', '/api/v2', 'GET')
    .addLink('users', '/api/v2/users', 'GET')
    .addLink('createUser', '/api/v2/users', 'POST')
    .addLink('products', '/api/v2/products', 'GET')
    .addLink('posts', '/api/v2/posts', 'GET')
    .addLink('auth', '/api/v2/auth/login', 'POST')
    .addLink('docs', '/api/v2/docs', 'GET')
    .addLink('health', '/health', 'GET');

  res.json(response);
});


router.get('/users', (req, res) => {
  res.json({
    _links: {
      self: { href: '/api/v2/users', method: 'GET' },
      create: { href: '/api/v2/users', method: 'POST' },
      find: { href: '/api/v2/users/{id}', method: 'GET',
templated: true },
      update: { href: '/api/v2/users/{id}', method: 'PUT',
templated: true },
```

```
      delete: { href: '/api/v2/users/{id}', method: 'DELETE',
templated: true }
    },
    query: {
      page: { type: 'number', default: 1 },
      limit: { type: 'number', default: 10 },
      sort: { type: 'string', example: '-createdAt' },
      filter: { type: 'object', example: '{"age": {"$gte":
18}}' }
    }
  });
});

module.exports = router;
```

6. Index Route with Authentication Status

javascript

```
const express = require('express');
const router = express.Router();
const auth = require('../middleware/auth');

router.get('/', auth.optional, async (req, res) => {
  const baseInfo = {
    api: 'MongoDB Express API',
    version: '1.0.0',
    timestamp: new Date().toISOString()
  };


  if (req.user) {
    const user = await User.findById(req.user.id).select('name
email role');

    res.json({
      ...baseInfo,
      welcome: `Welcome back, ${user.name}!`,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
      },
      yourResources: {
        profile: '/api/users/me',
        posts: '/api/users/me/posts',
        settings: '/api/users/me/settings'
      },
      recentActivity: {

      }
    });
  } else {

    res.json({
      ...baseInfo,
      message: 'Welcome to our API',
      status: 'public',
      authenticationRequired: ['POST /api/posts', 'PUT
/api/users/me', 'DELETE /api/posts/:id'],
      publicEndpoints: [
        'GET /api/users',
        'GET /api/products',
        'GET /api/posts',
        'POST /api/auth/login',
        'POST /api/auth/register'
      ]
```

```
    });
  }
});

module.exports = router;
```

7. Admin Dashboard Index Route

javascript

```
const express = require('express');
const router = express.Router();
const { auth, adminOnly } = require('../middleware/auth');

router.get('/', auth, adminOnly, async (req, res) => {
  try {

    const stats = {
      database: {
        collections: await
mongoose.connection.db.listCollections().toArray(),
        size: await mongoose.connection.db.stats(),
        indexes: {}
      },
      users: {
        total: await User.countDocuments(),
        active: await User.countDocuments({ isActive: true }),
        newToday: await User.countDocuments({
          createdAt: { $gte: new Date(new Date().setHours(0,
0, 0, 0)) }
        }),
        byRole: {
          admin: await User.countDocuments({ role: 'admin' }),
          user: await User.countDocuments({ role: 'user' })
        }
      },
      system: {
        uptime: process.uptime(),
        memory: process.memoryUsage(),
        cpu: process.cpuUsage(),
        nodeVersion: process.version,
        platform: process.platform
      },
      performance: {
        averageResponseTime: '120ms',
        requestsPerMinute: 1450,
        errorRate: '0.5%'
      }
    };


    const recentLogs = await Log.find()
      .sort({ createdAt: -1 })
      .limit(50);

    res.json({
      admin: {
        name: 'Admin Dashboard',
        email: req.user.email
```

```
      },
      statistics: stats,
      recentLogs,
      quickActions: {
        createUser: 'POST /api/admin/users',
        backupDatabase: 'POST /api/admin/backup',
        clearCache: 'POST /api/admin/cache/clear',
        systemHealth: 'GET /api/admin/health'
      },
      monitoring: {
        serverHealth: '/api/admin/health',
        metrics: '/api/admin/metrics',
        logs: '/api/admin/logs'
      }
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

8. Complete Server Setup with Index Routes

javascript

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();


const indexRoutes = require('./routes/index');
const userRoutes = require('./routes/users');
const productRoutes = require('./routes/products');
const authRoutes = require('./routes/auth');
const adminRoutes = require('./routes/admin');


app.use(express.json());
app.use(express.urlencoded({ extended: true }));


app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new
Date().toISOString()}`);
  next();
});


app.use('/', indexRoutes);
app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);
app.use('/api/auth', authRoutes);
app.use('/api/admin', adminRoutes);


app.use('/api/v1', indexRoutes);
app.use('/api/v2', require('./routes/index.v2'));


app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    database: mongoose.connection.readyState === 1 ?
'connected' : 'disconnected',
    uptime: process.uptime()
  });
});


app.use('*', (req, res) => {
  res.status(404).json({
    error: 'Route not found',
    message: `Cannot ${req.method} ${req.originalUrl}`,
```

```
    availableEndpoints: 'GET / for API documentation'
  });
});


app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    error: 'Internal Server Error',
    message: process.env.NODE_ENV === 'development' ?
err.message : 'Something went wrong'
  });
});


const PORT = process.env.PORT || 3000;
mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    console.log('✅ MongoDB connected');
    app.listen(PORT, () => {
      console.log(`🚀 Server running on
http://localhost:${PORT}`);
      console.log(`📚 API Documentation available at
http://localhost:${PORT}`);
    });
  })
  .catch(err => {
    console.error('❌ Database connection failed:', err);
    process.exit(1);
  });
```

Best Practices for Index Routes

✅ DO:

javascript

```
router.get('/', (req, res) => {
  res.json({
    version: '1.0.0',
    apiVersion: 'v1',
    documentation: '/api/v1/docs'
  });
});


router.get('/', (req, res) => {
  res.set('X-RateLimit-Limit', '100');
  res.set('X-RateLimit-Remaining', '95');
  res.json(apiInfo);
});


router.get('/', (req, res) => {
  res.set('Cache-Control', 'public, max-age=300');
  res.json(apiInfo);
});


router.get('/', (req, res) => {
  res.set('Access-Control-Allow-Origin', '*');
  res.json(apiInfo);
});
```

❌ DON'T:

javascript

```
router.get('/', (req, res) => {
  res.json({

    dbPassword: 'secret123',

    internalIP: '192.168.1.100',

    apiKey: process.env.SECRET_KEY
  });
});


router.get('/', async (req, res) => {

  const allUsers = await User.find();


  const userCount = await User.countDocuments();
});
```

Testing Index Routes

javascript

```
const request = require('supertest');
const app = require('../server');

describe('Index Routes', () => {
  test('GET / should return API info', async () => {
    const response = await request(app)
      .get('/')
      .expect(200);

    expect(response.body).toHaveProperty('name');
    expect(response.body).toHaveProperty('version');
    expect(response.body).toHaveProperty('endpoints');
  });

  test('GET /health should return server health', async () =>
{
    const response = await request(app)
      .get('/health')
      .expect(200);

    expect(response.body.status).toBe('healthy');
    expect(response.body).toHaveProperty('database');
  });

  test('GET /nonexistent should return 404', async () => {
    await request(app)
      .get('/nonexistent')
      .expect(404);
  });
});
```

Summary

The index route serves as the **entry point** and **documentation hub** for your API. A well-designed index route should:

1. **Provide clear API information** (name, version, description)
2. **List all available endpoints** with methods and descriptions
3. **Include examples** for common operations
4. **Show system status** (database connection, uptime)
5. **Link to detailed documentation**
6. **Follow REST conventions** and best practices

7. **Be fast and cacheable**

8. **Include helpful error messages**

Your index route is often the first thing developers see when exploring your API - make it informative and user-friendly!

new route all concept explain

Creating New Routes in Express with MongoDB - Complete Guide

What is a Route?

A **route** in Express determines how an application responds to client requests for specific endpoints (URIs) using specific HTTP methods (GET, POST, PUT, DELETE, etc.).

Route Structure Basics

javascript

```
app.METHOD(PATH, HANDLER);
```

1. Basic Route Creation

Simple Route Example

javascript

```
const express = require('express');
const app = express();


app.get('/api/hello', (req, res) => {
  res.json({ message: 'Hello World' });
});


app.post('/api/data', (req, res) => {
  res.json({ received: req.body });
});


app.put('/api/data/:id', (req, res) => {
  res.json({ updated: req.params.id });
});


app.delete('/api/data/:id', (req, res) => {
  res.json({ deleted: req.params.id });
});
```

2. Route Parameters

URL Parameters (req.params)

javascript

```
app.get('/api/users/:userId', (req, res) => {
  const userId = req.params.userId;
  res.json({ userId });
});


app.get('/api/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  res.json({ userId, postId });
});


app.get('/api/users/:id(\\d+)', (req, res) => {

  res.json({ userId: req.params.id });
});
```

Query Parameters (req.query)

javascript

```
app.get('/api/users', (req, res) => {
  const page = req.query.page || 1;
  const limit = req.query.limit || 10;
  const sort = req.query.sort || 'createdAt';

  res.json({ page, limit, sort });
});


app.get('/api/products', (req, res) => {
  const {
    category,
    minPrice,
    maxPrice,
    inStock,
    search
  } = req.query;


  const filter = {};
  if (category) filter.category = category;
  if (minPrice || maxPrice) {
    filter.price = {};
    if (minPrice) filter.price.$gte = Number(minPrice);
    if (maxPrice) filter.price.$lte = Number(maxPrice);
  }
  if (inStock === 'true') filter.inStock = true;

  res.json({ filter });
});
```

Body Parameters (req.body)

javascript

```
app.use(express.json());

app.post('/api/users', (req, res) => {
  const { name, email, age } = req.body;


  if (!name || !email) {
    return res.status(400).json({ error: 'Missing required
fields' });
  }

  res.json({ name, email, age });
});
```

3. Complete Route Implementation with MongoDB

User Routes - Full CRUD

javascript

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');




router.post('/users', async (req, res) => {
  try {
    const user = new User(req.body);
    const savedUser = await user.save();
    res.status(201).json({
      success: true,
      data: savedUser
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
});


router.post('/users/bulk', async (req, res) => {
  try {
    const users = await User.insertMany(req.body);
    res.status(201).json({
      success: true,
      count: users.length,
      data: users
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
});




router.get('/users', async (req, res) => {
  try {
    let query = User.find();


    if (req.query.isActive) {
```

```
      query =
query.where('isActive').equals(req.query.isActive === 'true');
    }

    if (req.query.minAge || req.query.maxAge) {
      const ageFilter = {};
      if (req.query.minAge) ageFilter.$gte =
parseInt(req.query.minAge);
      if (req.query.maxAge) ageFilter.$lte =
parseInt(req.query.maxAge);
      query = query.where('age', ageFilter);
    }


    if (req.query.search) {
      query = query.where({
        $or: [
          { name: { $regex: req.query.search, $options: 'i' }
},
          { email: { $regex: req.query.search, $options: 'i' }
}
        ]
      });
    }


    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;
    const skip = (page - 1) * limit;

    query = query.skip(skip).limit(limit);


    if (req.query.sort) {
      const sortField = req.query.sort.replace('-', '');
      const sortOrder = req.query.sort.startsWith('-') ? -1 :
1;
      query = query.sort({ [sortField]: sortOrder });
    }


    if (req.query.fields) {
      const fields = req.query.fields.split(',').join(' ');
      query = query.select(fields);
    }

    const users = await query;
    const total = await
User.countDocuments(query._conditions);

    res.json({
```

```
      success: true,
      count: users.length,
      total,
      pagination: {
        page,
        limit,
        totalPages: Math.ceil(total / limit),
        hasNext: page * limit < total,
        hasPrev: page > 1
      },
      data: users
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});


router.get('/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id)
      .select('-__v')
      .populate('posts', 'title content');

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});


router.get('/users/email/:email', async (req, res) => {
  try {
    const user = await User.findOne({ email: req.params.email
});
```

```
    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});




router.put('/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true,
        overwrite: true
      }
    );

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
```

```
  }
});


router.patch('/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true
      }
    );

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
});


router.patch('/users/:id/increment-age', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      { $inc: { age: 1 } },
      { new: true }
    );

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
```

```
    });
  }
});




router.delete('/users/:id', async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);

    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.json({
      success: true,
      message: 'User deleted successfully',
      data: user
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      error: error.message
    });
  }
});


router.delete('/users', async (req, res) => {
  try {
    const { ids } = req.body;
    const result = await User.deleteMany({ _id: { $in: ids }
});

    res.json({
      success: true,
      deletedCount: result.deletedCount,
      message: `${result.deletedCount} users deleted`
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
});
```

```
router.delete('/users/:id/soft', async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      { isActive: false, deletedAt: new Date() },
      { new: true }
    );

    res.json({
      success: true,
      message: 'User soft deleted',
      data: user
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    });
  }
});

module.exports = router;
```

4. Advanced Route Patterns

Nested Routes (Resource Relationships)

javascript

```
const express = require('express');
const router = express.Router({ mergeParams: true });
const Post = require('../models/Post');




router.get('/', async (req, res) => {
  try {
    const posts = await Post.find({ author: req.params.userId
});
    res.json({
      success: true,
      count: posts.length,
      data: posts
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});


router.post('/', async (req, res) => {
  try {
    const post = new Post({
      ...req.body,
      author: req.params.userId
    });
    await post.save();
    res.status(201).json({
      success: true,
      data: post
    });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});


router.get('/:postId', async (req, res) => {
  try {
    const post = await Post.findOne({
      _id: req.params.postId,
      author: req.params.userId
    });

    if (!post) {
      return res.status(404).json({ error: 'Post not found'
```

```
});
    }

    res.json({
      success: true,
      data: post
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

Using in main server:

javascript

```
const userRoutes = require('./routes/userRoutes');
const postRoutes = require('./routes/postRoutes');

app.use('/api', userRoutes);
app.use('/api/users/:userId/posts', postRoutes);
```

Route with Multiple HTTP Methods

javascript

```
const express = require('express');
const router = express.Router();

router.route('/products')

  .get(async (req, res) => {
    const products = await Product.find();
    res.json(products);
  })

  .post(async (req, res) => {
    const product = new Product(req.body);
    await product.save();
    res.status(201).json(product);
  })

  .delete(async (req, res) => {
    await Product.deleteMany({});
    res.json({ message: 'All products deleted' });
  });

router.route('/products/:id')

  .get(async (req, res) => {
    const product = await Product.findById(req.params.id);
    if (!product) return res.status(404).json({ error: 'Not
found' });
    res.json(product);
  })

  .put(async (req, res) => {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    res.json(product);
  })

  .patch(async (req, res) => {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(product);
  })

  .delete(async (req, res) => {
```

```
    await Product.findByIdAndDelete(req.params.id);
    res.json({ message: 'Product deleted' });
  });

module.exports = router;
```

5. Dynamic Routes with Controllers

Controller Pattern

javascript

```
class UserController {

  static async getAllUsers(req, res) {
    try {
      const users = await User.find();
      res.json({ success: true, data: users });
    } catch (error) {
      res.status(500).json({ success: false, error:
error.message });
    }
  }


  static async getUserById(req, res) {
    try {
      const user = await User.findById(req.params.id);
      if (!user) {
        return res.status(404).json({ success: false, error:
'User not found' });
      }
      res.json({ success: true, data: user });
    } catch (error) {
      res.status(500).json({ success: false, error:
error.message });
    }
  }


  static async createUser(req, res) {
    try {
      const user = new User(req.body);
      await user.save();
      res.status(201).json({ success: true, data: user });
    } catch (error) {
      res.status(400).json({ success: false, error:
error.message });
    }
  }


  static async updateUser(req, res) {
    try {
      const user = await User.findByIdAndUpdate(
        req.params.id,
        req.body,
        { new: true, runValidators: true }
      );
      if (!user) {
        return res.status(404).json({ success: false, error:
```

```
'User not found' });
      }
      res.json({ success: true, data: user });
    } catch (error) {
      res.status(400).json({ success: false, error:
error.message });
    }
  }


  static async deleteUser(req, res) {
    try {
      const user = await
User.findByIdAndDelete(req.params.id);
      if (!user) {
        return res.status(404).json({ success: false, error:
'User not found' });
      }
      res.json({ success: true, message: 'User deleted' });
    } catch (error) {
      res.status(500).json({ success: false, error:
error.message });
    }
  }
}

module.exports = UserController;
```

javascript

```


const express = require('express');
const router = express.Router();
const UserController =
require('../controllers/userController');

router.get('/users', UserController.getAllUsers);
router.get('/users/:id', UserController.getUserById);
router.post('/users', UserController.createUser);
router.put('/users/:id', UserController.updateUser);
router.delete('/users/:id', UserController.deleteUser);

module.exports = router;
```

6. Route Middleware

Custom Middleware for Routes

javascript

```
const jwt = require('jsonwebtoken');

const authMiddleware = async (req, res, next) => {
  try {
    const token = req.header('Authorization')?.replace('Bearer
', '');

    if (!token) {
      throw new Error();
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);

    if (!user) {
      throw new Error();
    }

    req.user = user;
    req.token = token;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Please authenticate' });
  }
};

const adminMiddleware = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access
required' });
  }
  next();
};

module.exports = { authMiddleware, adminMiddleware };
```

javascript

```
const express = require('express');
const router = express.Router();
const { authMiddleware, adminMiddleware } =
require('../middleware/auth');


router.get('/profile', authMiddleware, async (req, res) => {
  res.json({
    user: req.user,
    message: 'Access granted to protected route'
  });
});


router.get('/admin/users', authMiddleware, adminMiddleware,
async (req, res) => {
  const users = await User.find();
  res.json(users);
});


router.post('/posts',
  authMiddleware,
  validatePostInput,
  async (req, res) => {

  }
);

module.exports = router;
```

7. Route Validation

Input Validation Middleware

javascript

```javascript
const { body, param, query, validationResult } =
require('express-validator');

const validateUser = [
  body('name')
    .notEmpty().withMessage('Name is required')
    .isLength({ min: 2, max: 50 }).withMessage('Name must be
2-50 characters'),

  body('email')
    .isEmail().withMessage('Valid email is required')
    .normalizeEmail(),

  body('age')
    .optional()
    .isInt({ min: 0, max: 120 }).withMessage('Age must be
between 0 and 120'),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  }
];

const validateId = [
  param('id')
    .isMongoId().withMessage('Invalid ID format'),

  (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
    next();
  }
];

module.exports = { validateUser, validateId };
```

javascript

```
const router = require('express').Router();
const { validateUser, validateId } =
require('../middleware/validation');

router.post('/users', validateUser, async (req, res) => {

  const user = new User(req.body);
  await user.save();
  res.json(user);
});

router.get('/users/:id', validateId, async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
});
```

8. Route Versioning

javascript

```
app.use('/api/v1/users', require('./routes/v1/userRoutes'));
app.use('/api/v1/posts', require('./routes/v1/postRoutes'));


app.use('/api/v2/users', require('./routes/v2/userRoutes'));
app.use('/api/v2/posts', require('./routes/v2/postRoutes'));


app.use('/api/users', (req, res, next) => {
  const version = req.headers['api-version'] || '1.0';
  if (version.startsWith('2')) {
    return require('./routes/v2/userRoutes')(req, res, next);
  }
  return require('./routes/v1/userRoutes')(req, res, next);
});
```

9. Async Error Handling for Routes

javascript

```
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

module.exports = asyncHandler;


const asyncHandler = require('../utils/asyncHandler');

router.get('/users', asyncHandler(async (req, res) => {
  const users = await User.find();
  res.json(users);

}));
```

10. Complete Route File Structure

Modular Route Organization

text

```
routes/
├── index.js           # Main router
├── v1/
│   ├── userRoutes.js
│   ├── productRoutes.js
│   └── authRoutes.js
├── v2/
│   ├── userRoutes.js
│   └── productRoutes.js
├── middleware/
│   ├── auth.js
│   └── validation.js
└── api.js             # API router aggregator
```

api.js - Route Aggregator

javascript

```
const express = require('express');
const router = express.Router();


const userRoutes = require('./v1/userRoutes');
const productRoutes = require('./v1/productRoutes');
const authRoutes = require('./v1/authRoutes');


router.use('/users', userRoutes);
router.use('/products', productRoutes);
router.use('/auth', authRoutes);


router.get('/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date() });
});

module.exports = router;
```

server.js - Final Setup

javascript

```
const express = require('express');
const app = express();
const apiRoutes = require('./routes/api');

app.use(express.json());
app.use('/api', apiRoutes);


app.use('*', (req, res) => {
  res.status(404).json({ error: `Route ${req.originalUrl} not
found` });
});


app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: err.message });
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Route Best Practices Summary

✅ DO:

- Use meaningful route names (plural for resources)
- Implement proper HTTP methods for operations
- Validate input data
- Use route parameters for IDs, query for filters
- Implement pagination for list endpoints
- Use middleware for reusable logic (auth, validation)
- Handle errors gracefully
- Version your APIs
- Document your routes
- Use status codes appropriately

❌ DON'T:

- Don't use GET for state-changing operations
- Don't expose sensitive data in URLs
- Don't nest routes too deep (max 3 levels)
- Don't forget to handle 404 for invalid IDs
- Don't skip validation (even for trusted sources)
- Don't put business logic in route handlers
- Don't forget to return responses (avoid hanging requests)

This comprehensive guide covers everything you need to know about creating routes in Express with MongoDB!

create route all concept explain